Manual Testing Interview Questions & Answers

Here are some common manual testing interview questions and example answers:

1. **What is manual testing?**

Manual testing is the process of testing a software application manually, without the use of automated tools. It involves verifying that the software functions as expected and meets the specified requirements, by manually executing test cases and comparing the results to the expected outcomes.

2. **What is the difference between manual testing and automated testing?**

Manual testing is a testing method where the tester manually executes the test cases and compares the actual results to the expected results. Automated testing, on the other hand, is a testing method where the test cases are executed automatically using specialized software tools. Automated testing is generally faster and more efficient than manual testing, but it is also more expensive to set up and maintain.

3. **How do you plan and prepare for a manual testing project?**

There are several steps involved in planning and preparing for a manual testing project:

- **Understand the software requirements**: It is important to fully understand the requirements of the software being tested, including the functionality and features that need to be tested.
- **Identify the test scenarios**: Based on the software requirements, the test should identify the specific test scenarios that need to be covered. This may involve creating a list of test cases or a test plan.
- **Set up the testing environment**: The tester should ensure that the necessary testing environment is set up and configured properly. This may involve installing the software on a test machine,setting up test data, and configuring any necessary hardware or software components.
- **Select the appropriate testing tools**: Depending on the specific requirements of the project, the tester may need to select and configure appropriate testing toolsto assist with the testing process.

4. **How do you determine the test cases for a manual testing project?**

Test cases for a manual testing project can be determined in a number of ways, including:

- **Examining the software requirements**: The test cases should be based on the specific requirements and functionality of the software being tested.
- **Analyzing the software design**: The tester should examine the design of the software to identify any potential issues or areas that may need to be tested.
- **Identifying common scenarios**: The tester should consider the types of actions and scenarios that a user is likely to perform when using the software, and create test cases to cover these scenarios.
- **Using testing techniques such as boundary value analysis or equivalence partitioning**: These testing techniques involve dividing the input data into groups and testing the software with representative samples from each group.

5. **How do you report defects or issues found during manual testing?**

When a defect or issue is found during manual testing, it is important to report it clearly and accurately so that the development team can fix it. A good defect report should include:

- **A clear and concise description of the issue**: The tester should describe the problem in detail, including any error messages or unexpected behavior that was observed.
- **Steps to reproduce the issue**: The tester should provide clear, detailed steps that the development team can follow to reproduce the issue. Expected and actual results: The tester should describe the expected outcome of the test case, and compare it to the actual result that was observed.
- **Relevant information**: The tester should include any additional information that may be relevant to the issue,such as the version of the software being tested, the hardware and software configuration of the test machine, and any other relevant details.

6. **What is a test case?**

A test case is a set of conditions or variables under which a tester will determine whether a system under test satisfies requirements or works correctly. Test cases are typically used to validate that the system functions as expected, and to identify any defects or issues that may need to be addressed.

7. **How do you prioritize test cases?**

Test case prioritization is the process of deciding which test cases should be executed first, based on the importance or risk of the functionality being tested. There are several factors that can influence test case prioritization, including:

- **The impact of the functionality on the end user**: Test cases for high impact or mission-critical functionality should be given higher priority.
- **The likelihood of defects**: Test cases for functionality that has a history of defects or is known to be complex should be given higher priority.
- **The risk of the functionality**: Test cases for functionality that carries a high risk of failure or negative consequences should be given higher priority.

8. **How do you ensure the quality of your test cases?**

There are several ways to ensure the quality of test cases: Review the test cases with the development team: It can be helpful to review the test cases with the development team to ensure that they are comprehensive and cover all relevant scenarios.

- **Use peer review**: Test cases can be reviewed by other testers to ensure that they are clear, accurate, and complete.
- **Use testing templates**: Using a standardized template can help to ensure that all test cases follow a consistent format and include all necessary information.
- **Use test case management software**: Test case management software can help to organize and track test cases, and ensure that they are being executed consistently and accurately.

9. **How do you handle changes to the software during manual testing?**

It is common for changes to be made to the software during the testing process, either to fix defects or to add new features. When changes are made, it is important for the tester to re-test the affected areas to ensure that the changes

have not introduced any new defects. The tester should also update the test cases and any relevant documentation to reflect the changes.

10. **What are some challenges you have faced while doing manual testing, and how did you overcome them?**

Manual testing can present a number of challenges, including:

- **Maintaining the testing environmen**t: Ensuring that the testing environment is set up correctly and stays up to date can be a challenge, particularly if the software being tested has many dependencies or requires specific hardware or software configurations.
- **Managing test cases and documentation**: Keeping track of large numbers of test cases and related documentation can be time consuming and error-prone. Using test case management software or other tools can help to streamline this process.
- **Identifying and reproducing defects**: Sometimes defects can be difficult to identify or reproduce, especially if they are intermittent or only occur underspecific conditions. To overcome this challenge, the tester may need to use debugging tools or work closely with the development team to track down the root cause of the issue.

11. **What is exploratory testing?**

Exploratory testing is a type of testing that involves actively exploring the software to discover defects and learn more about the system. It is different from scripted testing, where the tester follows a predetermined set of test cases. Exploratory testing is often used to identify defects that may not have been anticipated in the planning stage, and to find new and creative ways to test the software.

12. **What are some benefits of using exploratory testing?**

There are several benefits to using exploratory testing, including:

- **Identifying defects that may not have been anticipated**: Exploratory testing can help to identify defects that may not have been identified through scripted testing, as the tester is free to explore the software in a more flexible and unstructured way.

- **Improving the understanding of the system**: Exploratory testing can help the tester to better understand the functionality and behavior of the software, which can be useful for identifying potential issues or areas for improvement.
- **Discovering new test cases**: Exploratory testing can lead to the discovery of new and creative test cases that may not have been identified through scripted testing.

13. **How do you document the results of exploratory testing?**

Documenting the results of exploratory testing can be challenging, as it is often more unstructured and open-ended than scripted testing. Some ways to document the results of exploratory testing include:

- **Taking detailed notes during the testing process**: The tester should document any observations, defects, or other important information as they are discovered.
- **Creating a report or summary of the testing**: After the testing is complete, the tester can create a report or summary that describes the testing process and any defects or issues that were identified.
- **Using test case management software**: Test case management software can be used to track and document the results of exploratory testing, including defects and any related documentation.

14. **What is a defect?**

A defect is an error, flaw, failure, or fault in the software that causes it to behave in an unexpected or undesired way. Defects can range from minor issues, such as typos or formatting errors, to more serious problems,such assystem crashes or security vulnerabilities.

15. **What is a defect lifecycle?**

The defect lifecycle, also known as the bug lifecycle, refers to the process of identifying, documenting, and resolving defects in the software. The stepsin the defect lifecycle typically include:

- **Identifying the defect**: The defect is identified and reported by the tester or user.

- **Prioritizing the defect**: The defect is prioritized based on the impact and severity of the issue.
- **Assigning the defect**: The defect is assigned to a developer or other team member for resolution.
- **Investigating the defect**: The developer investigates the root cause of the defect and determines the best course of action to fix it.
- **Fixing the defect**: The developer implements a fix for the defect.
- **Verifying the fix**: The tester verifies that the fix has resolved the defect and that the software is functioning correctly.
- **Closing the defect**: Once the defect has been fixed and verified, it is marked as closed and the process is complete.

16. **What is a test plan?**

A test plan is a document that outlines the testing strategy for a software project. It typically includes information such as the scope of the testing, the testing objectives, the test environment, and the resources and personnel needed for the testing process. The test plan serves as a roadmap for the testing process and helps to ensure that the testing is comprehensive and well-organized.

17. **What is the difference between a test case and a test scenario?**

A test case is a specific set of conditions or variables that are used to test the functionality of a system. It typically includes details such as the steps to follow, the expected outcome, and any relevant data or conditions. A test scenario is a description of a specific testing situation or scenario. It may include one or more test cases and can be used to test a particular aspect of the software or a specific user scenario.

18. **What are some best practices for writing test cases?**

Here are some best practices for writing test cases:

- **Make sure the test cases are clear and concise**: Test cases should be easy to understand and follow, with clear steps and expected outcomes.
- **Use descriptive and meaningful names for test cases**: Test case names should clearly and concisely describe the functionality being tested.

- **Include relevant details**: Test cases should include all relevant details, such as data values, conditions, and any necessary setup or cleanup steps.
- **Use standard templates**: Using a standard template can help to ensure that all test cases are consistent and include all necessary information.
- **Review and revise test cases as needed**: Test cases should be reviewed and revised as needed to ensure that they are accurate and up to date.

19. **What is a testsuite?**

A testsuite is a collection of test cases that are organized and grouped together to test a specific aspect of the software or a particular user scenario. Testsuites can be used to test a specific feature or module of the software, or to test the overall functionality of the system.

20. **What are the benefits of using testsuites?**

There are several benefits to using testsuites:

- **Improved organization and management**: Testsuites can help to organize and manage large numbers of test cases, making it easier to keep track of the testing process.
- **Reuse of test cases**: Test cases can be grouped into test suites and reused in multiple testing scenarios,reducing the time and effort needed to create new test cases.
- **Efficient execution of test cases**: Testsuites can be configured to run automatically, allowing the tester to efficiently execute large numbers of test cases.
- **Improved testing coverage**: Testsuites can help to ensure that the software is thoroughly tested, by grouping together test cases that cover a specific aspect of the system.

21. **What is a test data?**

Test data is the input data that is used to test the functionality of a system. It can include data values, files, and any other input that is needed to execute the test cases. Test data is used to verify that the system is functioning correctly and to identify any defects or issues.

22. **How do you select test data for manual testing?**

Test data should be selected carefully to ensure that the system is thoroughly tested and that any defects or issues are identified. Some factors to consider when selecting test data include:

- **The requirements and functionality of the system**: Test data should be selected to test the specific requirements and functionality of the system.
- **The types of input the system is expected to handle**: Test data should be selected to test the different types of input that the system is expected to handle,such as different data types or formats.
- **Boundary or edge cases**: Test data should be selected to test the limits or edge cases of the system,such as minimum and maximum values or invalid input.

23. **What is regression testing?**

Regression testing is the process of re-testing the software after changes have been made to ensure that the changes have not introduced any new defects or issues. Regression testing is important because changes to the software, even minor ones, can have unintended consequences and affect the overall functionality of the system.

24. **Why is regression testing important?**

Regression testing is important because it helps to ensure the stability and quality of the software. It helps to catch defects or issues that may have been introduced by changes to the software, and helps to ensure that the software continues to function as expected.

25. **How do you plan for regression testing?**

There are several steps involved in planning for regression testing:

- **Identify the areas of the software that have changed**: It is important to identify the specific areas of the software that have been changed, as these will need to be re-tested.
- **Select the test cases to be included in the regression testing**: The test cases that will be included in the regression testing should be selected based on the scope of the changes and the importance of the functionality being tested.

- **Create a regression testing schedule**: A regression testing schedule should be created to determine the sequence and timing of the regression testing.
- **Set up the testing environment**: The testing environment should be set up and configured as needed for the regression testing.
- **Execute the regression testing**: The selected test cases should be executed to verify that the changes have not introduced any new defects or issues.

26. **What is a bug?**

A bug is a defect or issue in the software that causes it to behave in an unexpected or undesired way. Bugs can range from minor issues,such as typos or formatting errors, to more serious problems,such as system crashes or security vulnerabilities.

27. **What is a bug report?**

A bug report is a document that describes a defect or issue in the software. It typically includes information such as the steps to reproduce the issue, the expected and actual results, and any relevant details or information about the issue. Bug reports are used to communicate defects to the development team and track the progress of their resolution.

28. **How do you report a bug?**

To report a bug, the tester should follow these steps:

- **Reproduce the issue**: The tester should be able to reproduce the issue consistently in order to report it accurately.
- **Gather relevant information**: The tester should gather as much information as possible about the issue, including any error messages, relevant data or conditions, and steps to reproduce the issue.
- **Create a bug report**: The tester should create a bug report that includes a clear and concise description of the issue, the steps to reproduce it, and any relevant information.
- **Submit the bug report**: The bug report should be submitted to the appropriate team or individual for review and resolution.

29. **What is a test environment?**

A test environment is the hardware and software configuration that is used to test the software. It can include the hardware and software components that are needed to run the software, as well as any necessary data or resources. The test environment should be set up and configured to match the production environment as closely as possible.

30. **What are the differences between a test environment and a production environment?**

A test environment is used specifically for testing purposes, while a production environment is the live environment in which the software is used by end users. Some differences between test and production environments include:

- **Hardware and software configuration**: The hardware and software configuration of the test environment may be different from the production environment, in order to test the software under different conditions.
- **Data**: The test environment may use test data that is different from the production data, in order to test the software with a variety of input values.
- **User access**: The test environment may have restricted user access, in order to ensure that the testing process is not disrupted.
- **Stability**: The test environment may not be as stable as the production environment, as it is used specifically for testing purposes.

31. **What is a smoke test?**

A smoke test is a quick and basic test of the software to ensure that it is working properly and that it is ready for further testing. Smoke tests are typically run after significant changes have been made to the software, or after a new build has been deployed.

32. **Why is a smoke test important?**

A smoke test is important because it helps to ensure that the software is stable and functioning correctly before more extensive testing is performed. If the smoke test fails, it may indicate that there are significant issues with the software that need to be addressed before further testing can be done.

33. **What are some common components of a smoke test?**

Common components of a smoke test include:

- **Basic functionality**: The smoke test should include tests of basic functionality, such as the ability to log in and navigate through the system.
- **Key features**: The smoke test should include tests of key features and functionality, such as the ability to create and edit or process transactions.
- **Performance**: The smoke tests should include basic tests of performance, such as the speed of the system and the response time of key features.

34. **What is a sanity test?**

A sanity test is a quick and basic test of the software to ensure that it is functioning correctly and that it is ready for further testing. Sanity tests are typically run after minor changes have been made to the software, or after a new build has been deployed.

35. **Why is a sanity test important?**

A sanity test is important because it helps to ensure that the software is stable and functioning correctly before more extensive testing is performed. If the sanity test fails, it may indicate that there are significant issues with the software that need to be addressed before further testing can be done.

36. **What is the difference between a smoke test and a sanity test?**

A smoke test is a quick and basic test of the software to ensure that it is working properly and that it is ready for further testing. It is typically run after significant changes have been made to the software, or after a new build has been deployed. A sanity test is a quick and basic test of the software to ensure that it is functioning correctly and that it is ready for further testing. It is typically run after minor changes have been made to the software, or after a new build has been deployed. The main difference between a smoke test and a sanity test is the scope and purpose of the testing. A smoke test is typically more comprehensive and covers a wider range of functionality, while a sanity test is more focused and checks only the most important features and functionality.

37. **What is a test summary report?**

A test summary report is a document that provides an overview of the testing process and the results of the testing. It typically includes information such as the scope of the testing, the test cases that were executed, the defects that were found, and the overall status of the testing. The test summary report is used to

communicate the results of the testing to stakeholders and to provide a summary of the testing process.

38. **What should be included in a test summary report?**

A test summary report should include the following information:

- **Overview of the testing process**: The report should provide an overview of the testing process, including the scope of the testing, the test cases that were executed, and the testing environment.
- **Results of the testing**: The report should include the results of the testing, including the number of defects found, the severity of the defects, and the overall status of the testing.
- **Conclusions and recommendations**: The report should include any conclusions or recommendations based on the results of the testing.
- **Appendices**: The report may include appendices with additional information,such as detailed test results or defect logs.

39. **What is a test closure report?**

A test closure report is a document that provides an overview of the testing process and the results of the testing, and formally closes the testing phase of the project.

40. **What is a test strategy?**

A test strategy is a document that outlines the overall approach to testing for a software project. It typically includes information such as the scope of the testing, the testing objectives, the types of testing to be performed, and the resources and personnel needed for the testing process. The test strategy serves as a roadmap for the testing process and helps to ensure that the testing is comprehensive and well organized.

41. **What are the different types of testing?**

There are several different types of testing that can be performed on software, including:

- **Unit testing**: Unit testing is a type of testing that focuses on individual units or components of the software. It is typically done by the

development team and is used to verify that each unit of the software is functioning correctly.

- **Integration testing**: Integration testing is a type of testing that focuses on verifying the interaction and integration between different components or modules of the software. It is used to ensure that the different components of the software are working together correctly.
- **System testing**: System testing is a type of testing that focuses on the overall functionality of the software as a system. It is used to verify that the software meets the specified requirements and performs as expected in a real-world environment.
- **Acceptance testing**: Acceptance testing is a type of testing that focuses on verifying that the software meets the needs and expectations of the end user. It is typically done by the end user or a representative of the end user, and is used to determine whether the software is ready for release.

42. **What is usability testing?**

Usability testing is a type of testing that focuses on the ease of use and user experience of the software. It is used to verify that the software is easy to use, intuitive, and meets the needs of the end user. Usability testing can be performed through a variety of methods,such as usability surveys, userinterviews, and usertesting sessions.

43. **What is compatibility testing?**

Compatibility testing is a type of testing that verifies that the software is compatible with different hardware,software, and operating systems. It is used to ensure that the software can be used on a variety of different platforms and devices without any issues.

44. **What is security testing?**

Security testing is a type of testing that focuses on verifying the security of the software. It is used to identify and address any vulnerabilities or weaknesses in the software that could be exploited by attackers. Security testing can include a variety of methods,such as vulnerability assessments, penetration testing, and security audits.

45. **What is performance testing?**

Performance testing is a type of testing that focuses on verifying the performance and scalability of the software. It is used to ensure that the software can handle the expected workload and userload without any issues. Performance testing can include a variety of methods, such as load testing,stresstesting, and endurance testing.

46. **What is exploratory testing?**

Exploratory testing is a type of testing that is focused on exploration and discovery. It involves testing the software in an unstructured way, with the goal of finding defects and learning more about the software. Exploratory testing is typically done by experienced testers who have a deep understanding of the software and its intended use.

47. **What is a test case template?**

A test case template is a standardized format for writing and organizing test cases. It typically includes fieldsforthe test case name, description,steps, expected results, and any relevant notes or information. Using a test case template can help to ensure that all test cases are consistent and include all necessary information.

48. **What is a test case management tool?**

A test case management tool is a software application that is used to organize, manage, and track the testing process. It can be used to create and organize test cases, manage defects and issues, and track the progress of the testing. Test case management tools can help to improve the efficiency and effectiveness of the testing process by providing a central location for all testing-related information and activities.

49. **What is a defect tracking tool?**

A defect tracking tool is a software application that is used to manage and track defects or issues in the software. It can be used to report, track, and resolve defects, as well as to communicate and coordinate the resolution of defects with different teams and stakeholders. Defect tracking tools can help to improve the efficiency and effectiveness of the defect resolution process by providing a central location for all defect-related information and activities.

50. **What is continuous testing?**

Continuous testing is a testing approach that involves the continuous

execution of automated tests as part of the software development process. It is typically done in conjunction with continuous integration and continuous delivery (CI/CD) practices, in order to ensure that the software is always in a state of readiness for deployment. Continuous testing helps to identify and address defects and issues early in the development process, and can improve the overall quality and reliability of the software.

51. **What is test automation?**

Test automation is the process of using software tools to automate the execution of test cases. Test automation can be used to speed up the testing process, reduce the amount of manual testing required, and improve the consistency and accuracy of the testing.

52. **What are the benefits of test automation?**

There are several benefits to using test automation, including:

1. **Increased efficiency**: Test automation can significantly reduce the amount of manual testing that is required, which can improve the efficiency of the testing process.
2. **Improved accuracy**: Test automation can help to improve the accuracy of the testing, as it reduces the risk of human error.
3. **Faster feedback**: Test automation can provide faster feedback on the status of the software, as it can run tests much more quickly than a human tester.
4. **Greater coverage**: Test automation can be used to test a wider range of functionality and scenarios, which can provide greater coverage of the software.

53. **What are some challenges of test automation?**

There are several challenges to using test automation, including:

1. **Initial setup and configuration**: Setting up and configuring the test automation tools and infrastructure can be time-consuming and require specialized skills.
2. **Maintenance**: Test automation scripts and frameworks need to be maintained and updated as the software changes, which can be an ongoing effort.
3. **Limited scope**: Test automation is most effective for testing repetitive or predictable scenarios, but may not be suitable for more complex or unstructured testing.
4. **Integration with other tools**: Test automation may need to be integrated with other tools and systems,such as defect tracking or test case management tools, which can add complexity to the process.

54. **What are some best practices for test automation?**

Some best practices for test automation include:

- **Define clear goals and objectives**: It is important to define clear goals and objectivesforthe test automation efforts, in orderto ensure that the automation is aligned with the overall testing strategy.
- **Select the right tools**: Careful consideration should be given to the selection of test automation tools, based on the needs and capabilities of the software and the team.
- **Create a robust test automation framework**: A robust test automation framework is essential for the success of the test automation efforts. The framework should be well-designed and maintainable, and should include strategies for handling complex or dynamic scenarios.
- **Use version control**: Test automation scripts and frameworks should be managed using version control, in order to track changes and ensure that the correct versions are being used.
- **Monitor and maintain the test automation**: Test automation requires ongoing maintenance and monitoring to ensure that it is running smoothly and providing the desired results.

55. **What is load testing?**

Load testing is a type of performance testing that involves testing the software under a simulated load or stress in order to verify its performance and scalability.

Load testing is typically used to verify that the software can handle the expected workload and userload without any issues.

56. **What is stress testing?**

Stress testing is a type of performance testing that involves testing the software under extreme or unrealistic conditions in order to verify its performance and robustness. Stress testing is typically used to identify the limits and weaknesses of the software, and to verify that it can handle unexpected or adverse conditions.

57. **What is endurance testing?**

Endurance testing is a type of performance testing that involves testing the software for an extended period of time in order to verify its performance and stability over time. Endurance testing is typically used to identify issues that may arise after prolonged use of the software,such as memory leaks orresource exhaustion.

58. **What is regression testing?**

Regression testing is a type of testing that is used to verify that changes or modifications to the software have not introduced any new defects or issues. Regression testing is typically performed after changes have been made to the software, and involves re-running previously successful test cases to ensure that the software is still functioning correctly.

59. **What is a test plan?**

A test plan is a document that outlines the approach and strategy for testing a software project. It typically includes information such as the scope of the testing, the testing objectives, the types of testing to be performed, the resources and personnel needed for the testing process, and the schedule for the testing. The test plan serves as a roadmap for the testing process and helps to ensure that the testing is comprehensive and well-organized.

60. **What is a test case?**

A test case is a document that describes a specific scenario or test condition to be tested, along with the expected results. Test cases are used to verify that the software is functioning correctly and meets the specified requirements. Each test

case typically includes a set of steps to be followed and the expected result of each step. Test cases are used to guide the testing process and ensure that all necessary functionality is tested.